

Python para Ciencia de la Atmósfera

- Clases:
 - 6 de Marzo 2026
 - 10 de Marzo 2026

—> Introducción a clases (class) de Python

Clases de python

- Una **clase** es un molde para crear objetos.
- Define qué datos tiene (**atributos**) y qué puede hacer (**métodos**).
- Los **atributos** son las variables que pertenecen a cada objeto. Cada objeto tiene sus propios datos independientes. Se acceden con **self.atributo** dentro de la clase y con **objeto.atributo** desde fuera.
- Los **métodos** son funciones que viven dentro de la clase. Siempre reciben **self** como primer parámetro, que es la referencia al objeto mismo. Sirven para operar sobre los atributos del objeto.

Sintaxis de la definición de clases

```
class ClassName:  
    <statement-1>  
    .  
    .  
    .  
    <statement-N>
```

- Una clase en Python se define con la palabra clave **class** seguida del nombre de la clase.
- El contenido de la clase suele incluir **funciones**, que dentro de una clase se llaman **métodos**.
- La definición de la clase debe ejecutarse primero para que la clase exista en el programa.
- Cuando Python entra en una definición de clase, crea un nuevo espacio de nombres donde se guardan sus variables y métodos.
- Al terminar la definición, Python crea un objeto de clase y lo asocia al nombre de la clase que se declaró.

Class objects

- Los **objetos de clase** en Python permiten dos operaciones principales:
 - Acceder a atributos
 - Crear instancias (objetos de la clase)

```
class MyClass:  
    """A simple example class"""  
    i = 12345  
  
    def f(self):  
        return 'hello world'
```

Class objects

- Una **instancia** es un objeto creado a partir de una clase. De una sola clase puedes crear muchos objetos, cada uno con datos diferentes. Esto es lo que hace útiles a las clases: defines la estructura una vez y la reutilizas.
- La **instanciación de una clase** utiliza la notación de función. Podemos imaginar que el objeto de la clase es una función sin parámetros que devuelve una nueva instancia de la clase.
- La operación de **instanciación** (es decir, “llamar” a un objeto de clase) crea un **objeto vacío**.
- Muchas clases prefieren crear objetos cuyas instancias estén **personalizadas con un estado inicial específico**. Por ello, una clase puede definir un **método especial llamado `__init__()`**.
- Cuando una clase define un método `__init__()`, la instanciación de la clase invoca automáticamente `__init__()` para la nueva instancia de la clase que se acaba de crear.

```
class Dog:

    kind = 'canine'          # class variable shared by all instances

    def __init__(self, name):
        self.name = name    # instance variable unique to each instance

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.kind                # shared by all dogs
'canine'
>>> e.kind                # shared by all dogs
'canine'
>>> d.name                # unique to d
'Fido'
>>> e.name                # unique to e
'Buddy'
```

```
class Dog:
```

[Copy](#)

```
    def __init__(self, name):
        self.name = name
        self.tricks = []    # creates a new empty list for each dog
```

```
    def add_trick(self, trick):
        self.tricks.append(trick)
```

```
>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks
['roll over']
>>> e.tricks
['play dead']
```

Ejercicios

1. Crea una clase llamada **Mascota** que al momento de crearla se **inicialicen automáticamente tres atributos: nombre, tipo y edad**. Luego crea tres mascotas diferentes e imprime sus atributos.

Ejercicios

1. Crea una clase llamada **Mascota** que al momento de crearla se **inicialicen automáticamente tres atributos: nombre, tipo y edad**. Luego crea tres mascotas diferentes e imprime sus atributos.
2. Crea una clase **Cancion** con atributos **titulo, artista y duracion_segundos**. Crea 3 canciones y calcula la duración total en minutos.
 1. Tip: crea una lista **canciones** de objetos Canciones
 2. Luego usa una comprensión de lista para obtener el atributo **duracion_segundos** de cada objeto.
 3. Calcula el tiempo total

Ejercicios

1. Crea una clase llamada **Mascota** que al momento de crearla se **inicialicen automáticamente tres atributos: nombre, tipo y edad**. Luego crea tres mascotas diferentes e imprime sus atributos.
2. Crea una clase **Canción** con atributos **titulo, artista y duracion_segundos**. Crea 3 canciones y calcula la duración total en minutos.
 1. Tip: crea una lista **canciones** de objetos Canciones
 2. Luego usa una comprensión de lista para obtener el atributo **duracion_segundos** de cada objeto.
 3. Calcula el tiempo total
3. Crea una clase **CuentaBancaria** que al crearla se inicialice con un **titular** obligatorio y un saldo que por defecto sea 0. Además, cada cuenta debe tener automáticamente una lista vacía de movimientos sin que el usuario la pase como parámetro. Crea dos cuentas con saldos diferentes. Modifica el saldo de una y demuestra que la otra no se ve afectada.

Ejercicios

1. Crea una clase llamada **Mascota** que al momento de crearla se **inicialicen automáticamente tres atributos: nombre, tipo y edad**. Luego crea tres mascotas diferentes e imprime sus atributos.
2. Crea una clase **Canción** con atributos **titulo, artista y duracion_segundos**. Crea 3 canciones y calcula la duración total en minutos.
 1. Tip: crea una lista **canciones** de objetos Canciones
 2. Luego usa una comprensión de lista para obtener el atributo **duracion_segundos** de cada objeto.
 3. Calcula el tiempo total
3. Crea una clase **CuentaBancaria** que al crearla se inicialice con un **titular** obligatorio y un saldo que por defecto sea 0. Además, cada cuenta debe tener automáticamente una lista vacía de movimientos sin que el usuario la pase como parámetro. Crea dos cuentas con saldos diferentes. Modifica el saldo de una y demuestra que la otra no se ve afectada.
4. Crea una clase **Jugador** con atributos **nombre, goles** (inicia en 0) y **partidos** (inicia en 0). Crea una lista de 5 jugadores y simula una temporada usando **random.randint()** para asignarles goles y partidos jugados. Imprime quién tiene mejor promedio de goles por partido.

Ejercicios

1. Crea una clase llamada **Mascota** que al momento de crearla se **inicialicen automáticamente tres atributos: nombre, tipo y edad**. Luego crea tres mascotas diferentes e imprime sus atributos.
2. Crea una clase **Canción** con atributos **titulo, artista y duracion_segundos**. Crea 3 canciones y calcula la duración total en minutos.
 1. Tip: crea una lista **canciones** de objetos Canciones
 2. Luego usa una comprensión de lista para obtener el atributo **duracion_segundos** de cada objeto.
 3. Calcula el tiempo total
3. Crea una clase **CuentaBancaria** que al crearla se inicialice con un **titular** obligatorio y un saldo que por defecto sea 0. Además, cada cuenta debe tener automáticamente una lista vacía de movimientos sin que el usuario la pase como parámetro. Crea dos cuentas con saldos diferentes. Modifica el saldo de una y demuestra que la otra no se ve afectada.
4. Crea una clase **Jugador** con atributos **nombre, goles** (inicia en 0) y **partidos** (inicia en 0). Crea una lista de 5 jugadores y simula una temporada usando **random.randint()** para asignarles goles y partidos jugados. Imprime quién tiene mejor promedio de goles por partido.
5. Crea una clase **Rectangulo** con atributos **base y altura**. Agrégale métodos **area()**, **perimetro()** y **es_cuadrado()** (devuelve True/False). Genera 10 rectángulos con dimensiones aleatorias y muestra cuántos son cuadrados.

__str__

- Cuando hacemos `print(objeto)`, Python muestra algo como `<__main__.Clase object at 0x...>`. El método `__str__` permite definir qué se muestra en su lugar.

Ejercicio 1: Crea una clase `Producto` con atributos `nombre` y `precio`. Haz que al usar `print()` sobre un producto se muestre automáticamente algo como `Café — $45.50` en vez del texto crudo del objeto.

Crea una lista de 3 productos e imprímelos con un `for`.

__str__

- Cuando hacemos `print(objeto)`, Python muestra algo como `<__main__.Clase object at 0x...>`. El método `__str__` permite definir qué se muestra en su lugar.

Ejercicio 1: Crea una clase `Producto` con atributos `nombre` y `precio`. Haz que al usar `print()` sobre un producto se muestre automáticamente algo como `Café — $45.50` en vez del texto crudo del objeto.

Crea una lista de 3 productos e imprímelos con un `for`.

Ejercicio 2: Crea una clase `Pelicula` con `titulo`, `director` y `calificacion` (del 1 al 10). Implementa `__str__` para que muestre algo como `8.5 | Inception (Christopher Nolan)`. Crea una lista de 5 películas e imprímelas ordenadas por calificación (de mayor a menor).

Pista: usa `sorted()` con `key=lambda p: p.calificacion`.

Ejercicio 3: Crea una clase `Dado` que por defecto tenga 6 caras, pero que pueda crearse con cualquier número de caras. Debe tener un método `lanzar()` que devuelva un resultado aleatorio y lo guarde en un historial, y un método `veces_salio(numero)` que diga cuántas veces ha salido un número. Crea un dado de 6 caras y uno de 20 caras. Lanza cada uno 100 veces y muestra cuántas veces salió el 6 en el primero y el 20 en el segundo.

__str__

- Cuando hacemos `print(objeto)`, Python muestra algo como `<__main__.Clase object at 0x...>`. El método `__str__` permite definir qué se muestra en su lugar.

Ejercicio 1: Crea una clase `Producto` con atributos `nombre` y `precio`. Haz que al usar `print()` sobre un producto se muestre automáticamente algo como `Café — $45.50` en vez del texto crudo del objeto.

Crea una lista de 3 productos e imprímelos con un `for`.

Ejercicio 2: Crea una clase `Pelicula` con `titulo`, `director` y `calificacion` (del 1 al 10). Implementa `__str__` para que muestre algo como `8.5 | Inception (Christopher Nolan)`. Crea una lista de 5 películas e imprímelas ordenadas por calificación (de mayor a menor).

Pista: usa `sorted()` con `key=lambda p: p.calificacion`.

Ejercicio 3: Crea una clase `Dado` que por defecto tenga 6 caras, pero que pueda crearse con cualquier número de caras. Debe tener un método `lanzar()` que devuelva un resultado aleatorio y lo guarde en un historial, y un método `veces_salio(numero)` que diga cuántas veces ha salido un número. Crea un dado de 6 caras y uno de 20 caras. Lanza cada uno 100 veces y muestra cuántas veces salió el 6 en el primero y el 20 en el segundo.

Ejercicio 4: Crea una clase `Sensor` con atributos `nombre` y `lecturas` (lista vacía). Métodos: `registrar(valor)` y `promedio()`. Simula 3 sensores (temperatura, humedad, presión) que registran 50 lecturas aleatorias cada uno. Imprime el promedio de cada sensor.

Herencia

- **Idea:**
- Una clase puede **heredar** de otra.
- La clase que hereda se llama **subclase** o **clase hija**, y la original se llama **superclase** o **clase padre**.

```
class DerivedClassName(BaseClassName):  
    <statement-1>  
    .  
    .  
    .  
    <statement-N>
```

Herencia

Sobrescribir métodos (override):

- Una clase hija puede modificar un método heredado.

Usar `super()`

- Sirve para llamar métodos de la clase padre.

Herencia múltiple

- Python permite heredar de varias clases.

```
class Empleado:
    pass
# se deben de inicializar nombre y salario. Son argumentos de __init__
# debe de haber un metodo mostrar_info, que imprima el nombre del empleado y el salario

class Desarrollador(Empleado): # esta heredando de la clase Empleado
    def __init__(self, nombre, salario, lenguaje: str):
        super().__init__(nombre, salario)
        self.lenguaje = lenguaje

# crear una instancia con Desarrollador
# imprimir su informacion

# hay que modificar Desarrollador para sobrescribir el método mostrar_info,
# que muestre solo el lenguaje
# usar super() para sobrescribir el metodo mostrar_info pero reutilizando el metodo del padre
# y añadir la info del lenguaje
```